Corelytics

Teljes dokumentáció

Készítették: Gubek-Szabó Tamás 13.I

Mészáros Imre 13.I

Szoftverspecifikáció

Tartalomjegyzék

1. Bevezetés ..... - 3 -
1.1 A Program címe ..... - 3 -
1.2 A program rövid ismertetése ..... - 3 -
2. Elvárások a programmal kapcsolatban ..... - 4 -
2.1 Operációs rendszer és környezet ..... - 4 -
2.2 Felhasználandó programozási nyelvek ..... - 4 -
2.3 A megoldás formátuma ..... - 4 -
2.4 Szoftverfejlesztés ..... - 4 -
3. A rendszer követelményei ..... - 5 -
3.1 Felhasználói fiók kezelésre ..... - 5 -
3.2 Étrend és edzésterv követése, szerkesztése ..... - 5 -
3.3 Google Health Applikáció csatlakoztatása ..... - 5 -

Corelytics

1. Bevezetés

1.1 A Program címe

Corelytics

1.2 A program rövid ismertetése

A Corelytics alkalmazás segítségével a testedzés és tréning nemcsak hatékonyabbá, hanem átláthatóbbá és motiválóbbá is válik. Az applikáció egyszerűen kezelhető, mégis professzionális eszközöket kínál a felhasználók számára, hogy nyomon követhessék teljesítményüket, és folyamatosan optimalizálhassák edzésterveiket. Így bárki könnyedén és tudatosan haladhat a saját sportcéljai felé, legyen szó állóképesség-növelésről, izomtömeg-gyarapításról vagy általános fittség megőrzéséről.

A felhasználó részletes statisztikákat és elemzéseket kap az edzéseiről, beleértve a teljesítmény alakulását, az edzésintenzitást és a regenerációs szinteket is. Ezek az adatok segítenek abban, hogy pontos képet kapjon a fejlődéséről, és időben be tudjon avatkozni, ha valamilyen területen stagnálás vagy túlterhelés jelei mutatkoznak. Emellett a Corelytics lehetőséget biztosít az edzések naplózására, személyes megjegyzések hozzáadására, sőt az eredmények megosztására is, akár edzővel, akár barátokkal, vagy közösségi platformokon keresztül.

A Corelytics így nem csupán egy technikai eszköz, hanem egy teljes körű, digitális edzőtárs, amely támogat a céljaid elérésében, miközben motivál és segít megőrizni a lendületedet a mindennapok során is.

2. Elvárások a programmal kapcsolatban

2.1 Operációs rendszer és környezet

- Windows 10, 11
- Android

2.2 Felhasználandó programozási nyelvek

- Capacitor (Mobile wrapper)
- Next.JS (Framework)
- Typescript (Type safety)
- Nextauth.JS (Login)
- PostgreSQL (Database)
- Prisma (Database Interactions)
- Tailwind CSS (Responsivity)
- tRPC (API validation)
- ZOD (Enviromental Variable Managment)
- TanStack query (Client State Managment)
- React-Hook-Form (Form Managment)
- Axios (Client API Calls)
- Vercel (Deployment)

2.3 A megoldás formátuma

1. PostgreSQL adatbázis kialakítása, formázás
2. Backend
    a. Regisztráció, bejelentkezés
    b. Adatbázis és program összekötése
    c. Pontos számítások
    d. Google Fit összekötés
3. Frontend
    a. Megjelenítés reszponzivitása

2.4 Szoftverfejlesztés

A feladat egy olyan webalapú alkalmazás fejlesztése, amely lehetőséget biztosít a felhasználók számára hogy jegyzeteljék a tréning tervüket, étrendjüket és a Google Fit applikáció összekötésével a statisztikáikat. A rendszernek biztosítania kell a felhasználók regisztrációját és egyszerű bejelentkezését hogy minden felhasználó saját profillal rendelkezzen, illetve a felhasználók adatainak biztonságát.

3. A rendszer követelményei

Az alábbiakban részletesen bemutatjuk a Corelytics rendszer legfontosabb funkcionális követelményeit. A rendszer három főbb részre épül: felhasználói fiók kezelésre, étrend és edzésterv követése ill. szerkesztése, Google Fit App csatlakozása

3.1 Felhasználói fiók kezelésre

A rendszer csak regisztrált és bejelentkezett felhasználóknak engedi az adatok feltöltését, szerkesztését és egyéb interakciókat. A fiók kezelés során ezeket a funkciókat kell biztosítani:

1. Regisztráció: A felhasználók e-mail címükkel és egy biztonságos jelszóval tudnak regisztrálni. A profil adja a lehetőséget a privát edzésterv és étrend jegyzetelésre, frissítésére és a Google Fit App csatlakozására.
2. Bejelentkezés és hitelesítés: A regisztrált felhasználók bejelentkezhetnek a rendszerbe a megadott hitelesítési adataikkal. A jelszavakat titkosított módon kell tárolni. A rendszer lehetőséget biztosít a jelszó visszaállítására. A felhasználói fiókkezelés célja egy biztonságos és hatékony hozzáférés a Corelytics minden funkciójához.
3. Profilkezelés: A felhasználók frissíthetik profiljukat, megváltoztathatják jelszavukat, frissíthetik adataikat, és beállíthatják értesítési preferenciáikat.

3.2 Étrend és edzésterv követése, szerkesztése

A Corelytics központi funkciója az étrend és edzésterv követése és energiabevitel naplózása, amelynek során a felhasználók egyszerűen és strukturáltan naplózhatják a napi tréningjüket és étkezéseiket.

1. Étrend létrehozása: A felhasználó egyszerűen tudja jegyzetelni a napi kalória és egyéb tápértékek számát, napi limitekkel és célokkal.
2. Vonalkód olvasó: Az egyszerű naplózás érdekében, vonalkód vagy QR kód olvasóval hozzáfér az alkalmazás a bizonyos étel tápértékéhez. Ha esetleg nem, akkor a felhasználó feltudja tölteni az új vonalkódót hogy a jövőben ő, és más felhasználó is tudja használni.
3. Edzésterv kezelő: Az szoftveren keresztül a felhasználó feltudja jegyezni a napi aktivitását(futás, munka, stb.) illetve a saját testnevelését. A napi edzésterv lista formájában jelenik meg.

3.3 Google Fit Applikáció csatlakoztatása

A Corelytics alkalmazás kompatibilis a Google Fit alkalmazással, ami lehetővé teszi a két platform közötti problémamentes kapcsolatot. Ez az integráció lehetőséget biztosít a Corelytics számára, hogy valós időben hozzáférjen és megjelenítsen egészségügyi adatokat. Ennek köszönhetően a felhasználók egy átfogó és naprakész képet kapnak egészségi állapotukról egyetlen helyen.

4. A program adatlapja

4.1. A program neve:

Corelytics

4.2. Felhasználandó programozási nyelvek

- Capacitor (Mobile wrapper)
- Next.JS (Framework)
- Typescript (Type safety)
- Nextauth.JS (Login)
- PostgreSQL (Database)
- Prisma (Database Interactions)
- Tailwind CSS (Responsivity)
- tRPC (API validation)
- ZOD (Enviromental Variable Managment)
- TanStack query (Client State Managment)
- React-Hook-Form (Form Managment)
- Axios (Client API Calls)
- Vercel (Deployment)

4.3 Operációs rendszer, környezet

- Windows 10, 11
- Android

4.4 Program készítők

- Gubek-Szabó Tamás
- Mészáros Imre

Folyamatábrák

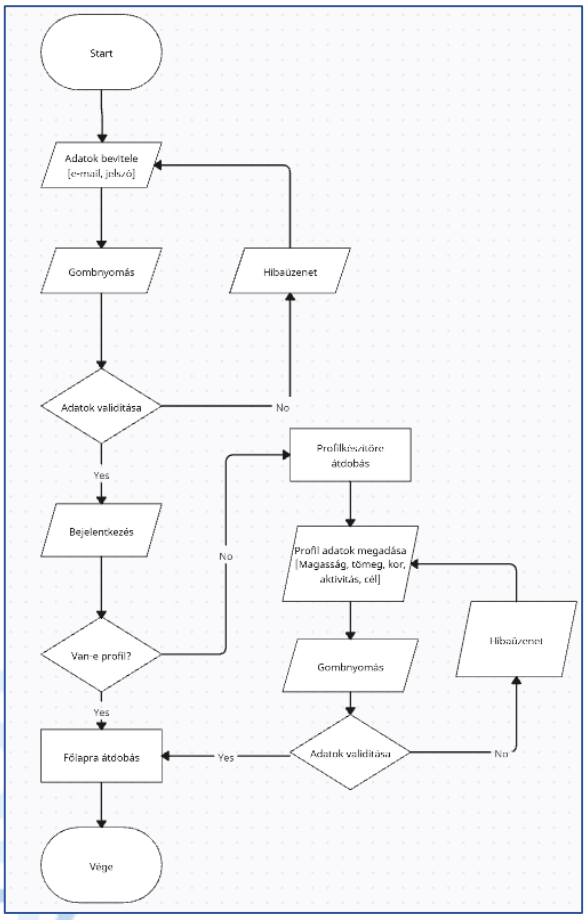
Tartalomjegyzék

1. Bejelentkezés ábrája ..... - 9 -
Folyamat magyarázata: ..... - 9 -
2. Regisztráció ábrája ..... - 10 -
Folyamat magyarázata: ..... - 10 -
3. Tápanyagszámítás folyamatábrája ..... - 11 -
Folyamat magyarázata ..... - 11 -
4. Étel hozzáadás ábrája ..... - 12 -
Folyamat magyarázata ..... - 12 -
5. Étkezés naplózás ábrája ..... - 13 -
Folyamat magyarázata ..... - 13 -

Corelytics

1. Bejelentkezés ábrája

A következő folyamatábra a Corelytics bejelentkezési funkciójának működését mutatja be.



Folyamat magyarázata:

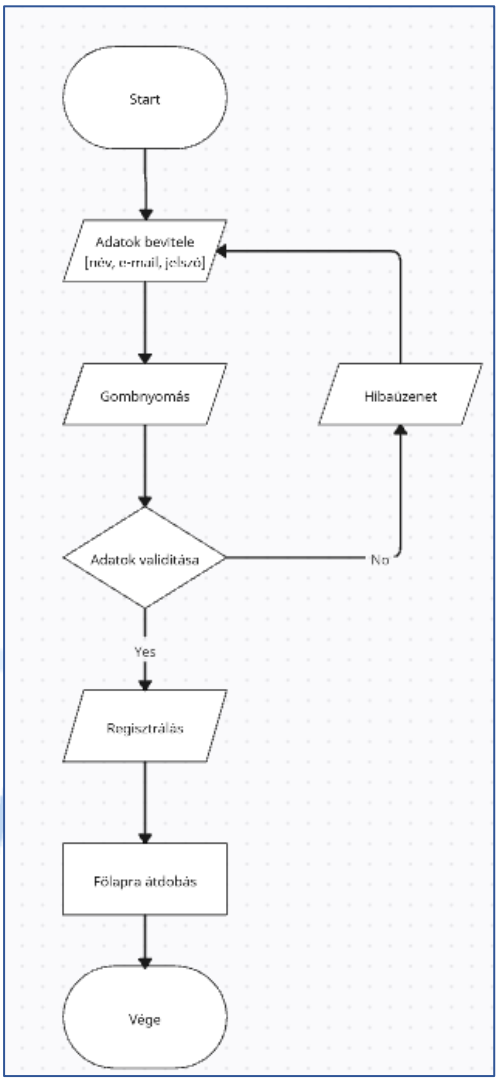
1. Start. A program elindul.
2. Adatok bevitele. Itt a bejelentkezési adatokat megadja a felhasználó, ami az e-mail és a hozzá csatolt jelszó
3. Gombnyomás, amikor a felhasználó megnyomja a bejelentkezés gombot
4. Adatok validitása. A program ellenőrzi hogy az adatok pontosak, létezik-e az adott e-maillel fiók, helyes-e a megadott jelszó.
    a. Ha hamis, azaz hibás, a program ki ad egy hibaüzenetet, ami vissza vezet az adatok bevitelére.
    b. Ha igaz akkor a program tovább folytatódik a Bejelentkezésre
5. Bejelentkezés. A program bejelentkezteti a felhasználót és hozzáférést ad minden funkcióhoz, aminek a feltétele egy bejelentkezett profil.
6. Van-e profil? Ellenőrzi a program hogy van e a user-hez userProfile illesztve az adatbázisban
    a. Ha nincs, azaz hamis, a felhasználót átirányítja a profilkészítőre
    b. Ha van, azaz igaz, átirányítja a felhasználót a főlapra
7. Profil adatok megadása. A profilkészítő lényege. A felhasználó megadja személyes adatait, ami a magasság, tömeg, kor, stb.
8. Gombnyomás. A felhasználó megnyomja a profil létre hozása gombot.
9. Adatok validitása. Le ellenőrzi a program hogy reális adatokat adott-e meg a felhasználó, például 300 cm és 50 cm között van, vagy 13 és 120 között van a kor.
    a. Ha hamis, azaz nem megfelelő adatokat adott meg a felhasználó, a program hibaüzenetet dob vissza, és visszairányít a profil adatok megadására.
    b. Ha igaz, azaz megfelelőek az adatok, akkor a felhasználót átdobja a főlapra.
10. Főlapra átdobás. A program a felhasználót átirányítja a főlapra.
11. Vége.

2. Regisztráció ábrája

A következő folyamatábra a Corelytics regisztráció funkciójának működését mutatja be.

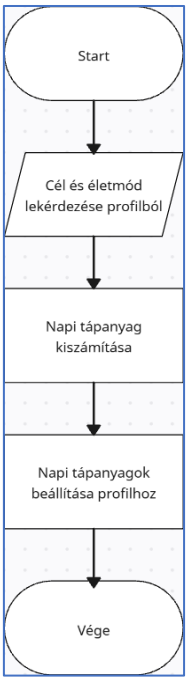
Folyamat magyarázata:

1. Start: A program elindul.
2. Adatok bevitele: A felhasználó beviszi a kívánt adatokat.
3. Gombnyomás: A felhasználó megnyomja a „Regisztrálás” gombot.
4. Adatok validitása: A rendszer ellenőrzi hogy az adatok megfelelnek-e az elvárásnak. Az elvárásokba beletartozik a érvényes e-mail cím, megfelelő jelszó.
    a. Ha hamis, azaz nem felelnek meg az adatok, akkor a feladat egy hibaüzenetet ad ki a felhasználónak, magyarázattal.
    b. Ha igaz, azaz megfelelnek, akkor a rendszer tovább megy a Regisztrálásra.
5. Regisztrálás: A felhasználó adatait feltölti az adatbázisba.
6. Főlapra átdobás: A felhasználót átirányítjuk a főlapra.
7. Vége.



3. Tápanyagszámítás folyamatábrája

A tápanyagszámítás itt, a tápanyaglimitekre és célokra utal. Ebben beletartozik a naponta maximum kalória, a fehérje cél és a maximum zsír. Míg maga az ábra egyszerűnek látszódhat, mégis a program egyik legfőbb funkciója, amit matematikával hozzunk létre.



Folyamat magyarázata

1. Start, elindul a program, meghívtuk a függvényt.
2. Cél és életmód lekérdezése profilból. A rendszer az adatbázisból lekérdezi a jelenlegi felhasználó célját, életmódját és az adatait, amiben beletartozik a magasság, tömeg és a kor. Lényegében a felhasználó teljes profilját. Ezek az adatok bármikor változhatnak.
3. Napi tápanyag kiszámítása. A rendszer egy megadott képlettel kiszámítja a tápanyagcélokat és limiteket. Mivel a lekérdezett adatok bármikor változhatnak, a rendszer a célokat és limiteket nem menti el, minden alkalommal kiszámolja.
4. Napi tápanyagok beállítása profilhoz. A rendszer megjeleníti a kiszámított adatokat a felhasználónak.
5. Vége.

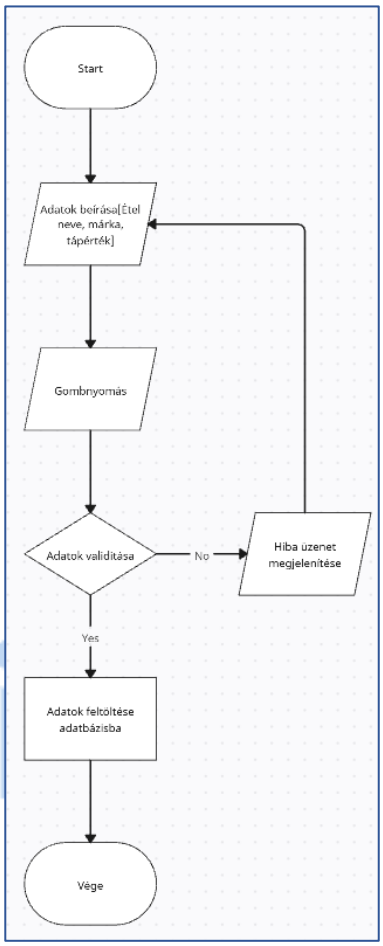
Corelytics

4. Étel hozzáadás ábrája

A következő ábra a felhasználók által létrehozott ételek folyamatábráját magyarázza el. Ennek a funkciónak köszönhetően lesz az adatbázis ételekkel feltöltve.

Folyamat magyarázata

1. Start, a folyamat elkezdődik.
2. Adatok bevitele, a felhasználó feltölti az adatokat, ebbe bele tartozik az étel tápértéke, neve illetve márkája. A név és márka alapján tudnak majd más felhasználók hivatkozni az ételre.
3. Gombnyomás, a felhasználó megnyomja a feltöltés gombot.
4. Adatok validálása, a rendszer ellenőrzi hogy megfelelnek e az adatok, illetve hogy minden ki lett e töltve.
    a. Ha hamis, azaz rosszak az adatok, a rendszer kiír egy hibaüzenetet oda, ahol hibát talált, majd a folyamat újra indul az adatok bevitelétől.
    b. Ha igaz, azaz az adatok megfelelőek, a folyamat tovább halad.
5. Adatok feltöltése az adatbázisba, a rendszer végül feltölti az adatokat az adatbázisba.
6. Vége.

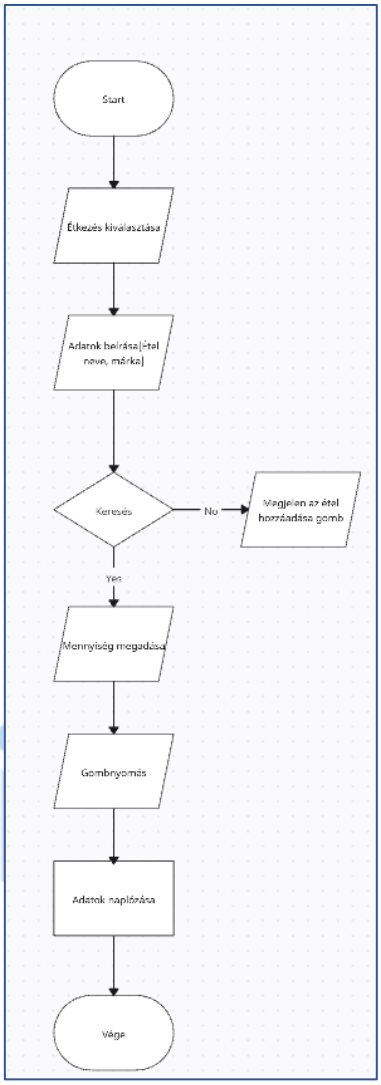


5. Étkezés naplózás ábrája

A Corelytics egyik legfőbb funkciója, ezzel tudják a felhasználók naplózni a bevitt kalóriát.

Folyamat magyarázata

1. Start, a folyamat elkezdődik.
2. Étkezés kiválasztása, a felhasználó kiválasztja hogy mikor evett, amibe beletartozik az ebéd, reggeli.
3. Adatok beírása, a felhasználó beírja az adatokat a keresőbe, ami lehet az étel neve, márkája.
4. Keresés, a rendszer átnézi az adatbázist és vissza dobja a találatokat
    a. Ha hamis, azaz nem talált semmit, felajánlja hogy a felhasználó adja hozzá az ételt.
    b. Ha igaz, azaz talált, akkor a felhasználó kitudja választani.
5. Mennyiség megadása. A felhasználó beírja mennyi adagot evett meg, és a rendszer ezzel szorozza meg az étel adatait.
6. Gombnyomás, a felhasználó megnyomja az étel naplózása gombot.
7. Adatok naplózása, a rendszer feltölti az adatbázisba tápanyagokat, illetve az elfogyasztott ételek azonosítóját.
8. Vége.



Adatbázis szerkezet

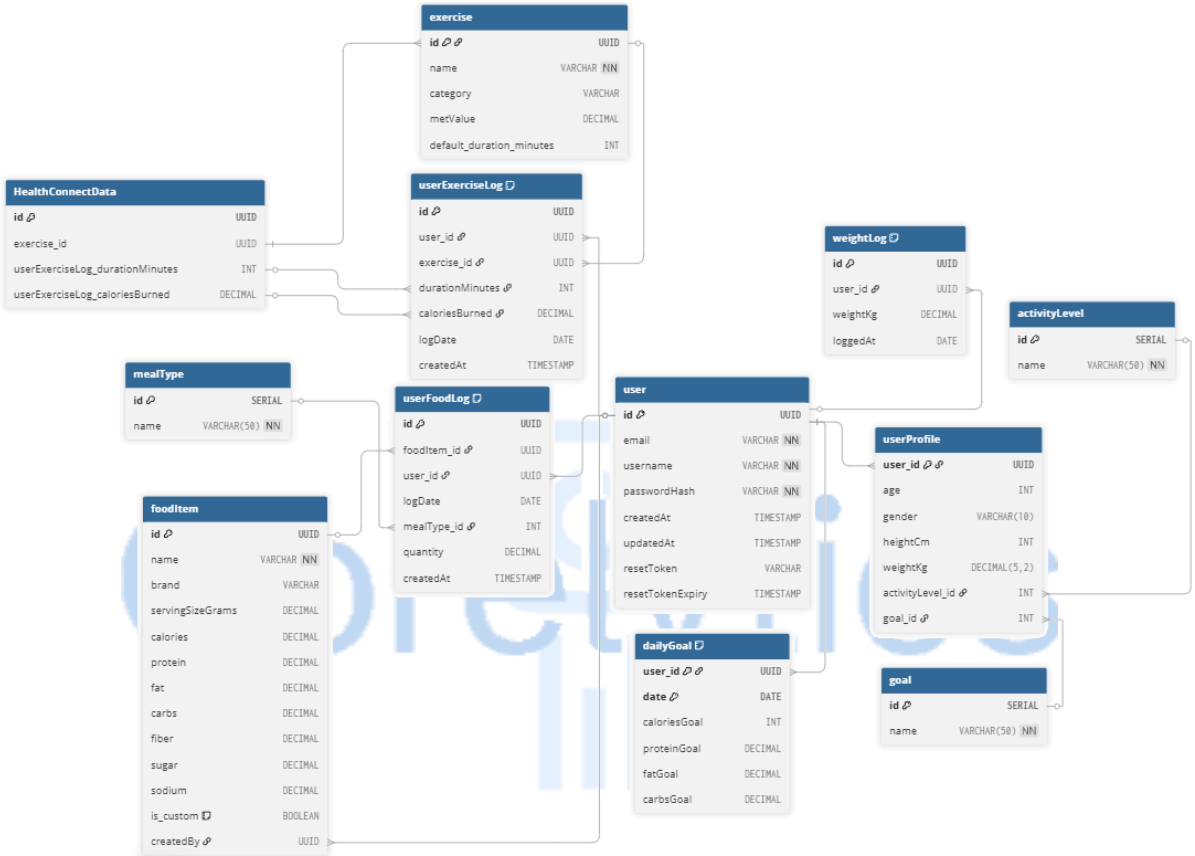
Tartalomjegyzék

1. A teljes adatbázis ..... - 16 -
2. „user” (Felhasználó) tábla ..... - 17 -
2. „userProfile” (Felhasználói profil) tábla ..... - 18 -
2.1. „activityLevel” (Életvitel/tevékenység) tábla: ..... - 18 -
2.2. „goal” (Cél) tábla: ..... - 18 -
2.3. „dailyGoal” (Napi cél) tábla: ..... - 18 -
2.4. „weightLog” (Tömeg napló) tábla: ..... - 19 -
3. „userFoodLog” (Felhasználói táplálék napló) ..... - 20 -
3.2. „foodItem” (Étel) tábla: ..... - 20 -
3.3. „mealType” (Étkezési típus) tábla: ..... - 21 -
4. „userExerciseLog” (Felhasználói testedzés napló) tábla ..... - 22 -
4.2. „exercise” (Gyakorlat) tábla ..... - 22 -
5. „HealthConnectData” (Health Connect adat) tábla ..... - 23 -

Corelytics

1. A teljes adatbázis

A Corelytics adatbázisának teljes szerkezete a lenti képen látható, a dokumentumban a különböző táblák részei lesznek elmagyarázva.



2. „user” (Felhasználó) tábla

A user tábla a Corelytics felhasználóinak az adatait tárolja el, amit használhat a bejelentkezéshez.

user	
id	UUID
email	VARCHAR NN
username	VARCHAR NN
passwordHash	VARCHAR NN
createdAt	TIMESTAMP
updatedAt	TIMESTAMP
resetToken	VARCHAR
resetTokenExpiry	TIMESTAMP

1. eleme: „id”:
   Az „id” elem minden felhasználóhoz tartozó különböző és automatikusan megadott UUID (Unique User ID). A UUID egy betűkből és számokból összeállított karakterlánc, ami a véletlenszerűsége miatt, egy extra védelmet nyújt a felhasználóknak. Ez az elsődleges kulcs.
1. eleme: „email”:
   Az „email” VARCHAR formátumban van eltárolva. A program ezt használja a különböző emailek elküldésére, illetve a bejelentkezéshez. Ez a mező nem lehet üres.
2. eleme: „username”:
   A „username” egy VARCHAR formátumban eltárolt adat. Ez a felhasználónév, amit a program használ a felhasználó megszólítására. Szintén nem lehet üres.
3. eleme: „passwordHash”:
   A „passwordHash” egy VARCHAR, ami a jelszót hash-ként tárol el. A hash szintet visszafejthetetlen, így hogyha valaki feltöri az adatbázist akkor nem fogja tudni a felhasználók jelszavát. Az egyetlen mód a kitalálására, hogyha ugyan azon jelszót hash-olunk le, ugyanis egy betű különbség is egy teljesen más karakter láncot ad vissza. Az adat nem lehet NULL.
4. eleme: „createdAt”:
   Egy időbélyeg, ami annyit jelent, hogy mikor jött létre a rekord.
5. eleme: „updatedAt”:
   Hasonló az előzőhez, csak nem a rekord létrejöttét, hanem a legutóbbi változását tárolja le.
6. eleme: „resetToken”:
   A „resetToken” egy VARCHAR, amit a program az „elfelejtett jelszó” funkcióhoz használ.
7. eleme: „resetTokenExpiry”:
   A „resetTokenExpiry” egy időbélyeg, ami az előző „resetToken” érvényességét határozza meg. A megadott időpont után a resetToken használhatatlan.

2. „userProfile” (Felhasználói profil) tábla

A userProfile tábla az, ahol a felhasználónak a személyes adati vannak eltárolva.

userProfile	
user_id	UUID
age	INT
gender	VARCHAR(10)
heightCm	INT
weightKg	DECIMAL(5,2)
activityLevel_id	INT
goal_id	INT

1. eleme: „user_id”:
   Egy UUID kulcs, ez köti össze a táblát a „user” táblával. Ez az elsődleges kulcs.
2. eleme: „age”:
   A felhasználó kora egy integerben eltárolva. Manuálisan változtatható a felhasználó által.
3. eleme: „gender”:
   A felhasználó neme, VARCHAR-ként eltárolva.
4. eleme: „heightCm”:
   A felhasználó magasága centiméterben, integerként tárolva.
5. eleme: „weightKg”:
   A felhasználó tömege, tizedes törtként tárolva, kilogrammban számolva.
6. eleme: „activityLevel_id”:
   Az életmód szintjének az IDja, összeköti a táblát az „activityLevel” táblával.
7. eleme: „goal_id”:
   A kitűzött cél IDja, összeköti a táblát a „goal” táblával

2.1. „activityLevel” (Életvitel/tevékenység) tábla:

Az activityLevel tábla a különböző életvitel típusokat tárolja le.

activityLevel	
id	SERIAL
name	VARCHAR(50) NN

1. eleme: „id”:
   Az „id” egy SERIAL adat, ami egy automatikusan megadott integer. Ez az elsődleges kulcs. Összekötésben áll az activityLevel_id-val a userProfile táblából.
2. eleme: „name”:
   Az életvitel neve, VARCHAR formátumban. Ezt használja a program, hogy kiszámolja a bizonyos célokhoz szükséges tápanyagbevitelt. A cella nem lehet üres.

2.2. „goal” (Cél) tábla:

A goal tábla a különböző célokat tárolja le.

goal	
id	SERIAL
name	VARCHAR(50) NN

1. eleme: „id”:
   Hasonló az activityLevel táblában használt id-hoz, de ez a goal_id-val áll összekötésben. Ez az elsődleges kulcs.
2. eleme: „name”:
   A megadott cél neve, VARCHAR-ként eltárolva. A program ezt használja a tápanyaglimit számításhoz. Nem lehet üres a cella.

2.3. „dailyGoal” (Napi cél) tábla:

A dailyGoal a napi tápanyag limitet határozza meg.

dailyGoal	
user_id	UUID
date	DATE
caloriesGoal	INT
proteinGoal	DECIMAL
fatGoal	DECIMAL
carbsGoal	DECIMAL

1. eleme: „user_id”:
   UUID-ként tárolt adat. Összeköti a felhasználóval. Ez az elsődleges kulcs.

2. eleme: „date”:
   Egy időpontként tárolt adat. Bizonyos napokra van osztva.
3. eleme: „caloriesGoal”:
   Egy integer, amit a program számol ki a felhasználónak az adataitól függően. Az adat kcal mértékegységben van tárolva.
4. eleme: „proteinGoal”:
   A napi fehérje beviteli cél, tizedes törtként tárolja a rendszer. Az adatot a program határozza meg.
5. eleme: „fatGoal”:
   Napi zsír limitje, tizedes törtként tárolva. A tárolt számot a program határozza meg.
6. eleme: „carbsGoal”:
   Napi szénhidrát limit. A tárolt tizedes törtet a program határozza meg.

2.4. „weightLog” (Tömeg napló) tábla:

A felhasználó tömegének naplózása, amit a felhasználó tölt fel manuálisan.

weightLog	
id	UUID
user_id	UUID
weightKg	DECIMAL
loggedAt	DATE

1. eleme: „id”:
   A naplónak az azonosítója, UUID-ként megadva. Ez a mező az elsődleges kulcs.
2. eleme: „user_id”:
   Egy UUID adat, amit a user táblától kap, a felhasználónak az azonosítója.
3. eleme: „weightKg”:
   A megadott tömeg kilogrammban, tizedes törtként tárolva.
4. eleme: „loggedAt”:
   A dátum amikor a tömeget a felhasználó naplózta.

3. „userFoodLog” (Felhasználói táplálék napló)

A userFoodLog tábla az adott felhasználók táplálék bevitelének naplózását biztosítja.

userFoodLog	
id	UUID
foodItem_id	UUID
user_id	UUID
logDate	DATE
mealType_id	INT
quantity	DECIMAL
createdAt	TIMESTAMP

1. eleme: „id”:
   Az id egy UUID-ként van eltárolva. Ez az elsődleges kulcs.
2. eleme: „foodItem_id”:
   A foodItem tábla id mezőjével áll összekötésben. Szintén UUID.
3. eleme: „user_id”:
   Az adott felhasználóval köti össze, UUID-ként tárolt.
4. eleme: „logDate”:
   A naplózás dátuma. Arra a napra utal amikor a naplózás megtörtént nem, amikor a rekord létrejött. Ez az adat egy időpontként van letárolva.
5. eleme: „mealType_id”:
   Az étkezésnek a típusa, például: reggeli, ebéd, vacsora. Ez egy integerként van eltárolva.
6. eleme: „quantity”:
   Az ételnek a mennyisége, grammként tárolva egy tizedes törtként.
7. eleme: „createdAt”:
   A rekord létrejöttének dátuma, időbélyegként tárolva

3.2. „foodItem” (Étel) tábla:

A foodItem tábla a bizonyos ételeket tárolja el.

foodItem	
id	UUID
name	VARCHAR NN
brand	VARCHAR
servingSizeGrams	DECIMAL
calories	DECIMAL
protein	DECIMAL
fat	DECIMAL
carbs	DECIMAL
fiber	DECIMAL
sugar	DECIMAL
sodium	DECIMAL
is_custom	BOOLEAN
createdBy	UUID

1. eleme: „id”:
   Az étel azonosítója, UUID-ként tárolva. Ez az elsődleges kulcs
2. eleme: „name”:
   A megadott étel neve, VARCHAR formátumban. Például rizs, kenyér, sonka. Nem lehet üres a cella.
3. eleme: „brand”:
   A rekord márkája, mivel több ételből különböző márkák vannak. Például Maggi, Pick.
4. eleme: „servingSizeGrams”:
   Egy adagnak a gramm mennyisége, tizedes törtként megadva.
5. eleme: „calories”:
   A rekord kalória értéke, kcal mértékben, tizedes tört.
6. eleme: „protein”:
   Az étel fehérje mértéke, tizedes törtként megadva.
7. eleme: „fat”:
   Az étel zsírtartalma, tizedes tört.
8. eleme: „carbs”:
   Az étel szénhidrát tartalma. Tizedes törtként tárolt.
9. eleme: „fiber”:
   A megadott ételnek a rost tartalma, tizedes törtként tárolva.
10. eleme: „sugar:”
    Az étel cukortartalma, tizedes tört.

11. eleme: „sodium”:
    Az ételnek a só tartalma, szintén tizedes tört.
12. eleme: „is_custom”:
    Logikai változó, hogyha hamis akkor alapból megadott a fejlesztők által, hogyha igaz akkor egy felhasználó hozta létre a rekordot.
13. eleme: „createdBy”:
    Egy UUID, amely összeköti a rekord létrehozójának azonosítójával. Hogyha a fejlesztők által megadott, akkor NULL.

3.3. „mealType” (Étkezési típus) tábla:

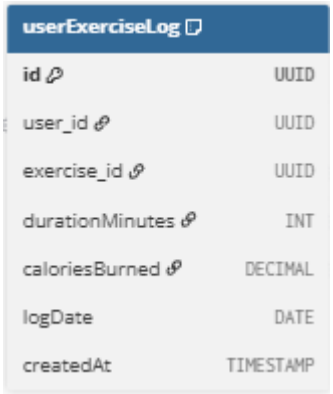
Az adott étkezésnek a típusait tárolja el.

mealType	
id	SERIAL
name	VARCHAR(50) NN

1. eleme: „id”:
   Egy SERIAL-ban megadott azonosító, amire a „mealType_id” hivatkozik a userFoodLog táblában. Ez az elsődleges kulcs.
2. eleme: „name”:
   Egy VARCHAR-ban tárolt adat, az étkezési típus neve, például: vacsora. Nem lehet üres.

4. „userExerciseLog” (Felhasználói testedzés napló) tábla

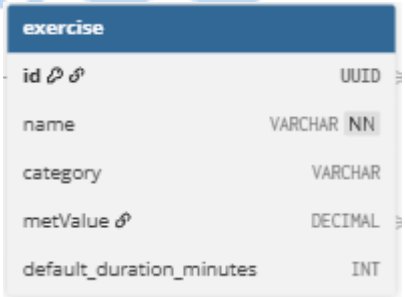
A felhasználónak az aznapi edzésének naplózására használt tábla.



1. eleme: „id”:
   A naplózott rekord azonosítója, UUID-ként tárolva. Ez az elsődleges kulcs.
2. eleme: „user_id”:
   A felhasználó azonosítója, kapcsolatban áll a user tábla id mezőjével. UUID formátumban tárolt.
3. eleme: „exercise_id”:
   Az adott gyakorlatnak az azonosítója, mivel minden külön edzés típus külön táblában van tárolva. UUID a formátuma.
4. eleme: „durationMinutes”:
   Az edzés időtartama, integerként tárolva.
5. eleme: „caloriesBurned”:
   Az edzés által becsült égetett kalória értéke, tizedes törtként tárolva.
6. eleme: „logDate”:
   Az edzés megtörténésének dátuma, nem feltétlen egy napon naplózta a felhasználó az edzését.
7. eleme: „createdAt”:
   A naplózás létrejötte, időbélyegként tárolva. Annak a pontos időpontja, amikor a rekord létre lett hozva.

4.2. „exercise” (Gyakorlat) tábla

A rendszer által tárolt gyakorlatoknak a táblája.



1. eleme: „id”:
   A gyakorlatnak az azonosítója, összekötésben áll a userExerciseLog „exercise_id” mezőjével. Ez az elsődleges kulcs, amit UUID-ként van tárolva.
2. eleme: „name”:
   Az adott gyakorlatnak a neve, VARCHAR formátumban tárolva. Nem lehet NULL.
3. eleme: „category”:
   A gyakorlat kategóriája, legyen az erőépítés, kardió vagy hajlékonyság. VARCHAR-ként tárolt.
4. eleme: „metValue”:
   A gyakorlat által égetett kalória óránként. Tizedes törtként van tárolva.
5. eleme: „default_duration_minutes”:
   A gyakorlatnak alapból beállított ideje, integerként letárolva.

5. „HealthConnectData” (Health Connect adat) tábla

Az az adatok táblája, amelyet a program az Health Connect applikációtól kap.

HealthConnectData	
id	UUID
exercise_id	UUID
userExerciseLog_durationMinutes	INT
userExerciseLog_caloriesBurned	DECIMAL

1. eleme: „id”:
   A rekordnak az azonosítója, UUID formátumban mentve. Elsődleges kulcs.
2. eleme: „exercise_id”:
   A gyakorlatnak az azonosítója, UUID-ként tárolva. Összekötésben áll az exercise tábla id rekordjával.
3. eleme: „userExerciseLog_durationMinutes”:
   A felhasználói testedzési naplóban feltölti a gyakorlat idejét. Integerként tárolt.
4. eleme: „userExerciseLog_caloriesBurned”:
   Tizedes törtként megadott szám, amely meghatározza, hogy mennyi kalóriát égetett a felhasználó gyakorlat közben. Hogyha üres, a program az alap kalória égetéssel tölti fel az adatot.

Corelytics

Tesztesetek Magyarázata

A tesztek Robot Framework-el lettek automatizálva.

1. Regisztráció Teszt

Ez a teszt egy új felhasználó regisztrációját ellenőrzi.

Lépések:
- Megnyitja a böngészőt a megadott weboldalon.
- A navigációs sávban a bejelentkezés/regisztráció menüpontra kattint.
- Átvált a regisztrációs oldalra.
- Kitölti a név, email, jelszó és jelszó megerősítése mezőket.
- Elküdli az űrlapot.

Ellenőrzés:
- A rendszer sikeres regisztráció után a bejelentkezési oldalra irányít.
- A teszt ellenőrzi, hogy megjelenik-e a „Jelentkezz be a fiókodba” szöveg.

Cél:
Annak validálása, hogy az új felhasználó sikeresen tud regisztrálni.

Teszt adat:	Várt eredmény	Kapott eredmény
Név: Teszt Elek	Név elfogadva	Név elfogadva
Email: Tesztelek@gmail.com	Email elfogadva	Email elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
Jelszó meg:12345678	A felhasználó regisztrált	A felhasználó regisztrált

2. Regisztráció Teszt – Létező felhasználó

Ez a teszt azt ellenőrzi, mi történik, ha már létező email címmel próbálnak regisztrálni.

Lépések:
- Megnyitja a weboldalt.
- A regisztrációs oldalra navigál.
- Ugyanazokat az adatokat adja meg, mint egy már létező felhasználó.

Ellenőrzés:
- Az űrlap nem irányít át másik oldalra.
- Az oldal tartalmazza továbbra is a regisztrációs formot.

Cél:
Annak ellenőrzése, hogy a rendszer megfelelően kezeli a duplikált regisztrációt.

Teszt adat:	Várt eredmény	Kapott eredmény
Név: Teszt Elek	Név elfogadva	Név elfogadva
Email: Tesztelek@gmail.com	Email nem elfogadva	Email nem elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
Jelszó meg:12345678	A felhasználó nem tud regisztrálni.	A felhasználó nem tud regisztrálni.

Corelytics

3. Bejelentkezés Teszt

Ez a teszt a meglévő felhasználó bejelentkezési folyamatát vizsgálja.

Lépések:
- Megnyitja a weboldalt.
- Rákattint a bejelentkezés menüpontra.
- Megadja az email címet és jelszót.
- Elküldi a bejelentkezési űrlapot.

Ellenőrzés:
- Sikeres bejelentkezés után a rendszer a „Profil kiegészítése” oldalra navigál.
- A teszt ellenőrzi a megfelelő fejléc szöveg megjelenését.

Cél:
Annak validálása, hogy a felhasználó sikeresen be tud jelentkezni.

Teszt adat:	Várt eredmény	Kapott eredmény
Email: Tesztelek@gmail.com	Email elfogadva	Email elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
	A felhasználó bejelentkezett.	A felhasználó bejelentkezett.

Corelytics

4. Profil Kiegészítése Teszt

Ez a teszt a profil kiegészítés funkció működését ellenőrzi.

Lépések:
- Bejelentkezik egy meglévő felhasználóval.
- Kitölti a születési dátum, nem, magasság, testsúly mezőket.
- Kiválasztja az aktivitási szintet és célt.
- Elküldi az űrlapot.

Ellenőrzés:
- Sikeres mentés után a felhasználó neve megjelenik a felső menüben.

Cél:
Annak ellenőrzése, hogy a profiladatok mentése megfelelően működik.

Teszt adat:	Várt eredmény	Kapott eredmény
Email: Tesztelek@gmail.com	Email elfogadva	Email elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
Szül dátum: 2007-01-10	Dátum elfogadva	Dátum elfogadva
Nem: férfi	Nem elfogadva	Nem elfogadva
Magasság: 170	Magasság elfogadva	Magasság elfogadva
Testsúly: 70	Testsúly elfogadva	Testsúly elfogadva
	A profil kieg. sikeres	A profil kieg. sikeres.

5. Étel naplózás Teszt

Ez a teszt az étel naplózás funkció működését vizsgálja.

Lépések:
- Bejelentkezik a rendszerbe.
- Navigál az étel naplózó oldalra.
- Új étel hozzáadását kezdeményezi.
- Beírja az étel nevét (Banán).
- Kiválasztja és hozzáadja az ételt a naplóhoz.

Ellenőrzés:
- A teszt ellenőrzi, hogy a „Banán” szöveg megjelenik-e a napi étellistában.

Cél:
Annak validálása, hogy az étel sikeresen hozzáadható a napi naplóhoz.

Teszt adat:	Várt eredmény	Kapott eredmény
Email: Tesztelek@gmail.com	Email elfogadva	Email elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
Étel: Banán	Étel létezik, elfogadva	Étel létezik, elfogadva
Adag: 2	Reális adag, elfogadva	Reális adag, elfogadva
	Ételt sikeresen naplóztuk	Ételt sikeresen naplóztuk

6. Felhasználó törlése Teszt

Ez a teszt a felhasználó törlés funkció működését vizsgálja

Lépések:
- Bejelentkezik a rendszerbe.
- Navigál a profil beállításokba
- Az oldal aljára teker
- Rákattint a profil törlése gombra és konfirmáció gombra

Ellenőrzés:
- A teszt ellenőrzi, hogy a felhasználó képes-e kitörölni a saját felhasználóját

Cél:
Annak validálása, hogy a felhasználó törlés funkció működőképes-e.

Teszt adat:	Várt eredmény	Kapott eredmény
Email: Tesztelek@gmail.com	Email elfogadva	Email elfogadva
Jelszó:12345678	Jelszó elfogadva	Jelszó elfogadva
	A felhasználó sikeresen törölve	A felhasználó sikeresen törölve.

Corelytics

Backend tesztelés

Automatizált egységtesztek dokumentációja

Tartalomjegyzék

1. Összefoglalás
2. Teszt kategóriák áttekintése
3. Validációs tesztek
    1. Környezeti változók séma (envSchema)
    2. Profil validációs séma (profileSchema)
    3. Étkezési napló validáció (foodLog)
    4. Étel létrehozás validáció (foodCreate)
4. API végpont tesztek
    1. Regisztráció (POST /api/auth/register)
    2. Elfelejtett jelszó (POST /api/auth/forgot-password)
    3. Jelszó visszaállítás (POST /api/auth/reset-password)
    4. Profil kezelés (GET & POST /api/profile)
    5. Aktivitási szintek és célok
    6. Fiók deaktiválás (POST /api/account/deactivate)
5. tRPC eljárás tesztek
    1. user.profile eljárás
6. Segédfüggvény tesztek
    1. calculateAge – Életkor számítás
    2. calculateDailyGoals – Napi célok számítás
7. Komponens tesztek (UI)
    1. Header komponens
    2. Footer komponens
    3. Főoldal (Home)
    4. Bejelentkezés oldal (SignIn)
    5. Regisztráció oldal (SignUp)
8. Összesített statisztika

1. Összefoglalás

Ez a dokumentum a **Corelytics** alkalmazás automatizált egységtesztjeinek teljes dokumentációját tartalmazza. Az alkalmazás egy MyFitnessPal-szerű fitness és táplálkozás követő rendszer, amely Next.js keretrendszerre épül TypeScript nyelven.

A tesztek a **Jest** teszt keretrendszert és a **React Testing Library**-t használják, és az alábbi területeket fedik le:

Tesztelt területek:
- **Validációs sémák** – Zod alapú bemeneti validáció tesztelése
- **API végpontok** – REST API route-ok egységtesztelése
- **tRPC eljárások** – Szerveroldali üzleti logika tesztelése
- **Segédfüggvények** – Számítási algoritmusok tesztelése
- **UI komponensek** – React komponensek renderelési és interakciós tesztelése

Összes tesztfájl	17 fájl
Összes teszteset	186 teszteset
Validációs tesztek	78 teszteset
API tesztek	43 teszteset
tRPC tesztek	21 teszteset
Segédfüggvény tesztek	24 teszteset
Komponens tesztek	50 teszteset

2. Teszt kategóriák áttekintése

Kategória	Fájlok száma	Tesztek száma	Leírás
Validáció (Zod)	4	78	Bemeneti adatok validációs sémáinak tesztelése
API végpontok	6	43	REST API route handlerek tesztelése mock-olt adatbázissal
tRPC eljárások	1	21	Szerveroldali tRPC eljárások üzleti logikájának tesztelése
Segédfüggvények	1	24	Számítási algoritmusok (BMR, TDEE, makrók) tesztelése

UI komponensek	5	50	React komponensek renderelési és interakciós tesztelése

#	Teszt csoport	Teszteset neve	Leírás	Elvárt eredmény
1	DATABASE_URL	érvényes PostgreSQL URL-t elfogad	Helyes PostgreSQL connection string megadása	**Sikeres validáció**
2	DATABASE_URL	elutasítja a hiányzó DATABASE_URL-t	DATABASE_URL mező hiányzik az objektumból	**Sikertelen validáció**
3	DATABASE_URL	elutasítja az érvénytelen URL-t	Nem URL formátumú szöveg megadása	**Sikertelen validáció**
4	DATABASE_URL	elutasítja az üres DATABASE_URL-t	Üres string megadása DATABASE_URL-ként	**Sikertelen validáció**
5	NEXTAUTH_URL	érvényes URL-t elfogad	HTTP localhost URL megadása	**Sikeres validáció**
6	NEXTAUTH_URL	HTTPS URL-t is elfogad	HTTPS production URL megadása (Vercel)	**Sikeres validáció**
7	NEXTAUTH_URL	elutasítja az érvénytelen NEXTAUTH_URL-t	Nem URL formátumú szöveg megadása	**Sikertelen validáció**
8	NEXTAUTH_URL	elutasítja a hiányzó NEXTAUTH_URL-t	NEXTAUTH_URL mező hiányzik	**Sikertelen validáció**

9	NEXTAUTH_SECRET	érvényes titkos kulcsot elfogad	Érvényes titkos kulcs string megadása	**Sikeres validáció**
10	NEXTAUTH_SECRET	elutasítja az üres NEXTAUTH_SECRET-et	Üres string megadása titokként	**Sikertelen validáció**
11	NEXTAUTH_SECRET	elutasítja a hiányzó NEXTAUTH_SECRET-et	NEXTAUTH_SECRET mező hiányzik	**Sikertelen validáció**
12	NEXTAUTH_SECRET	egy karakteres titkot is elfogad	Egyetlen karakter megadása titokként (min 1)	**Sikeres validáció**
13	NODE_ENV	elfogadja a development értéket	NODE_ENV = 'development'	**Sikeres, érték: development**
14	NODE_ENV	elfogadja a production értéket	NODE_ENV = 'production'	**Sikeres, érték: production**
15	NODE_ENV	elfogadja a test értéket	NODE_ENV = 'test'	**Sikeres, érték: test**
16	NODE_ENV	elutasítja az érvénytelen NODE_ENV értéket	NODE_ENV = 'staging' (nem engedélyezett)	**Sikertelen validáció**
17	NODE_ENV	alapértelmezetten development	NODE_ENV nincs megadva	**Sikeres, alapérték: development**

18	Teljes validáció	elfogad egy teljesen érvényes konfigurációt	Minden mező helyes értékkel	**Sikeres validáció**
19	Teljes validáció	elutasít egy teljesen üres objektumot	Üres objektum megadása	**Sikertelen validáció**
20	Teljes validáció	jelzi az összes hiányzó mezőt	Üres objektum – minden kötelező mező hiányzik	**Hibajelzés 3 mezőre**
21	Teljes validáció	figyelmen kívül hagyja az extra mezőket	Extra EXTRA_VAR mező az objektumban	**Sikeres validáció**

3. Validációs tesztek

A validációs tesztek a **Zod** könyvtár segítségével definiált bemeneti sémákat ellenőrzik. Ezek biztosítják, hogy az alkalmazás csak érvényes adatokat dolgozzon fel.

3.1 Környezeti változók séma (envSchema.test.ts)

Fájl: `src/__tests__/validation/envSchema.test.ts`

Cél: Az alkalmazás környezeti változóinak (DATABASE_URL, NEXTAUTH_URL, NEXTAUTH_SECRET, NODE_ENV) validációjának tesztelése Zod séma segítségével.

3.2 Profil validációs séma (profileSchema.test.ts)

Fájl: `src/__tests__/validation/profileSchema.test.ts`

Cél: A felhasználói profil űrlap adatainak (életkor, nem, magasság, súly, aktivitási szint, cél) validációjának tesztelése.

#	Teszt csoport	Teszteset neve	Leírás	Elvárt eredmény

1	Életkor	Valid kort el kell fogadni	Érvényes életkor (25) megadása	**Sikeres, érték: 25 (szám)**
2	Életkor	Üres kort el kell utasítani	Üres string megadása életkorként	**Hiba: "Age is required"**
3	Életkor	13 alatti kort el kell utasítani	12 éves kor megadása (minimum 13)	**Hiba: "Age must be at least 13"**
4	Életkor	120 feletti kort el kell utasítani	121 éves kor megadása (maximum 120)	**Hiba: "Age must be less than 120"**
5	Életkor	Nem szám kort el kell utasítani	Szöveges érték ("twenty") megadása	**Hiba: "Age must be a valid number"**
6	Nem	Valid nemet el kell fogadni	male, female, other értékek tesztelése	**Mind sikeres**
7	Nem	Nem értelmezhező nemet el kell utasítani	"invalid" nem érték megadása	**Hiba: "Please select a valid gender"**
8	Nem	Üres nemet el kell utasítani	Üres string megadása nemként	**Hiba: "Gender is required"**
9	Magasság	Valid magasságot el kell fogadni	175 cm megadása	**Sikeres, érték: 175 (szám)**
10	Magasság	50cm alatt el kell utasítani	49 cm megadása (minimum 50)	**Hiba: "Height must be at least 50cm"**
11	Magasság	300cm felett el kell utasítani	301 cm megadása (maximum 300)	**Hiba: "Height must be less than 300cm"**

12	Súly	Valid súlyt el kell fogadni	70.5 kg megadása (tizedes szám)	**Sikeres, érték: 70.5 (szám)**
13	Súly	20kg alatti súlyt el kell utasítani	19 kg megadása (minimum 20)	**Hiba: "Weight must be at least 20kg"**
14	Súly	500kg feletti súlyt el kell utasítani	501 kg megadása (maximum 500)	**Hiba: "Weight must be less than 500kg"**
15	Aktivitási szint	Valid ID-t el kell fogadni	Érvényes ID (3) megadása	**Sikeres, érték: 3 (szám)**
16	Aktivitási szint	Üres aktivitási szintet el kell utasítani	Üres string megadása	**Hiba: "Activity level is required"**
17	Aktivitási szint	Nem szám aktivitási szintet el kell utasítani	"high" szöveg megadása	**Hiba: "Please select a valid activity level"**
18	Cél	Érvényes ID-t el kell fogadni	Érvényes ID (2) megadása	**Sikeres, érték: 2 (szám)**
19	Cél	Üres célt el kell utasítani	Üres string megadása	**Hiba: "Fitness goal is required"**
20	Cél	Nem szám célt el kell utasítani	"lose_weight" szöveg megadása	**Hiba: "Please select a valid fitness goal"**

3.3 Étkezési napló validáció (foodLog.test.ts)

Fájl: `src/__tests__/validation/foodLog.test.ts`

Cél: Az étkezési napló műveleteihez (naplózás, módosítás, törlés, keresés, napi lekérdezés) tartozó validációs sémák tesztelése.

logFoodSchema – Étel naplózás

#	Teszteset neve	Leírás	Elvárt eredmény
1	should validate correct food log data	Helyes adatok megadása (foodItemId, mealTypeId, quantity, logDate)	**Sikeres validáció**
2	should validate food log without optional logDate	logDate opcionális mező nélkül	**Sikeres validáció**
3	should reject missing foodItemId	foodItemId mező hiányzik	**Sikertelen validáció**
4	should reject missing mealTypeId	mealTypeId mező hiányzik	**Sikertelen validáció**
5	should reject missing quantity	quantity mező hiányzik	**Sikertelen validáció**
6	should reject zero quantity	quantity = 0 (pozitívnak kell lennie)	**Sikertelen validáció**
7	should reject negative quantity	quantity = -5	**Sikertelen validáció**
8	should accept decimal quantities	quantity = 0.5 (tizedes szám)	**Sikeres validáció**
9	should accept large quantities	quantity = 999.99 (nagy érték)	**Sikeres validáció**

updateLogQuantitySchema – Mennyiség frissítés

#	Teszteset neve	Leírás	Elvárt eredmény

1	should validate correct update quantity data	Helyes logId és quantity megadása	**Sikeres validáció**
2	should reject missing logId	logId mező hiányzik	**Sikertelen validáció**
3	should reject missing quantity	quantity mező hiányzik	**Sikertelen validáció**
4	should reject zero or negative quantity	quantity = 0	**Sikertelen validáció**
5	should accept decimal quantities	quantity = 1.5	**Sikeres validáció**

deleteLogSchema – Napló törlés

#	Teszteset neve	Leírás	Elvárt eredmény
1	should validate correct delete log data	Helyes logId megadása	**Sikeres validáció**
2	should reject missing logId	logId mező hiányzik	**Sikertelen validáció**
3	should reject empty logId	Üres string megadása logId-ként	**Sikertelen validáció**

searchFoodSchema – Étel keresés

#	Teszteset neve	Leírás	Elvárt eredmény
1	should validate correct search data	Keresési szöveg és limit megadása	**Sikeres validáció**

2	should use default limit when not provided	Limit nélküli keresés	**Sikeres, limit = 20**
3	should reject empty query	Üres keresési szöveg	**Sikertelen validáció**
4	should reject missing query	query mező hiányzik	**Sikertelen validáció**
5	should accept single character query	Egy karakteres keresés ("a")	**Sikeres validáció**
6	should accept long query strings	Hosszú keresési szöveg	**Sikeres validáció**

getDailyLogsSchema – Napi naplók lekérdezés

#	Teszteset neve	Leírás	Elvárt eredmény
1	should validate with date provided	Dátum megadásával	**Sikeres validáció**
2	should validate without date (optional)	Dátum nélkül (opcionális)	**Sikeres validáció**
3	should accept various date formats	ISO 8601 formátumú dátum	**Sikeres validáció**

3.4 Étel létrehozás validáció (foodCreate.test.ts)

Fájl: `src/__tests__/validation/foodCreate.test.ts`

Cél: Az egyéni étel létrehozásának validációja – névkötelezőség, tápanyagértékek nem-negativitása, tizedes értékek kezelése.

#	Teszt csoport	Teszteset neve	Leírás	Elvárt eredmény

1	Alap	should validate complete food data with all fields	Minden mező kitöltve (név, márka, tápanyagok)	**Sikeres**
2	Alap	should validate food with only required name field	Csak kötelező név megadása	**Sikeres**
3	Alap	should validate food with name and some optional fields	Név + néhány opcionális mező	**Sikeres**
4	Név	should reject empty name	Üres név megadása	**Sikertelen**
5	Név	should reject missing name	Név mező hiányzik	**Sikertelen**
6	Tápanyag	should reject negative calories	Negatív kalória (-100)	**Sikertelen**
7	Tápanyag	should accept zero calories	0 kalória (víz)	**Sikeres**
8	Tápanyag	should reject negative protein	Negatív fehérje	**Sikertelen**
9	Tápanyag	should accept zero protein	0 fehérje	**Sikeres**
10	Tápanyag	should reject negative fat	Negatív zsír	**Sikertelen**
11	Tápanyag	should reject negative carbs	Negatív szénhidrát	**Sikertelen**
12	Tápanyag	should reject negative fiber	Negatív rost	**Sikertelen**

13	Tápanyag	should reject negative sugar	Negatív cukor	**Sikertelen**
14	Tápanyag	should reject negative sodium	Negatív nátrium	**Sikertelen**
15	Adag	should reject zero or negative serving size	0 gramm adagméret	**Sikertelen**
16	Adag	should accept positive serving size	100 gramm adagméret	**Sikeres**
17	Speciális	should accept decimal values for macros	Tizedes értékek (0.25, 0.65 stb.)	**Sikeres**
18	Speciális	should accept very large values	Nagy értékek (5000 kcal)	**Sikeres**
19	Speciális	should accept long food names	Hosszú ételnév	**Sikeres**
20	Speciális	should accept long brand names	Hosszú márkanév	**Sikeres**
21	Valós eset	should validate realistic complete meal example	Házi csirkesaláta teljes adatokkal	**Sikeres**
22	Valós eset	should validate zero-calorie items	Diétás üdítő (0 kcal)	**Sikeres**
23	Valós eset	should validate high-protein foods	Tejsavó fehérjepor	**Sikeres**

24	Valós eset	should validate high-fat foods	Extra szűz olívaolaj	**Sikeres**
25	Valós eset	should validate high-carb foods	Főtt fehér rizs	**Sikeres**
26	Valós eset	should handle foods with high sodium	Szójaszósz (magas nátrium)	**Sikeres**
27	Valós eset	should handle foods with high fiber	Chia mag (magas rost)	**Sikeres**
28	Típus	should reject invalid data types for numeric fields	Szöveges érték szám helyett	**Sikertelen**
29	Logika	should allow foods where macro calories exceed total	Makrók összege > megadott kalória (nincs keresztvalidáció)	**Sikeres (séma nem validál)**
30	Logika	should allow missing some macros even if calories are specified	Kalória megadva, de hiányzó makrók	**Sikeres**

4. API végpont tesztek

Az API tesztek a Next.js route handler-eket tesztelik mock-olt Prisma adatbázissal és NextAuth munkamenettel. Node.js környezetben futnak (`@jest-environment node`).

4.1 Regisztráció – POST /api/auth/register (register.test.ts)

Fájl: `src/__tests__/api/register.test.ts`

Cél: Felhasználó regisztrációs végpont tesztelése – új felhasználó létrehozás, duplikált email/felhasználónév kezelés, inaktív fiók újraaktiválás.

#	Teszteset neve	Leírás	Elvárt HTTP státusz	Elvárt válasz

1	sikeresen létrehoz egy új felhasználót	Érvényes email, jelszó és név megadása	201	"Felhasználó sikeresen létrehozva"
2	hibát ad vissza, ha az email már foglalt	Már létező aktív email cím	400	"Ezzel az email címmel már létezik felhasználó"
3	hibát ad vissza, ha a felhasználónév már foglalt	Már létező aktív felhasználónév	400	"Ez a felhasználónév már foglalt"
4	újraaktiválja az inaktív felhasználót	Inaktív fiókkal regisztráció ugyanazzal az email-lel	201	"Fiók sikeresen újraaktiválva"
5	a jelszó hashelt formában kerül tárolásra	bcrypt.hash meghívásának ellenőrzése	—	passwordHash = hashed érték
6	UUID-t generál az új felhasználónak	uuid.v4 használatának ellenőrzése	—	id = generált UUID
7	500-as hibát ad vissza adatbázis hiba esetén	Adatbázis kapcsolati hiba szimuláció	500	"Hiba a felhasználó létrehozása során"

4.2 Elfelejtett jelszó – POST /api/auth/forgot-password (forgotPassword.test.ts)

Fájl: `src/__tests__/api/forgotPassword.test.ts`

Cél: Jelszó-visszaállító email küldés végpont tesztelése – biztonsági válasz, token generálás, email küldés.

#	Teszteset neve	Leírás	Elvárt HTTP státusz	Elvárt válasz

1	sikeres választ ad létező felhasználó esetén	Létező email megadása	200	Sikeres üzenet
2	reset tokent ment az adatbázisba	Token és lejárat mentésének ellenőrzése	—	resetToken + resetTokenExpiry frissítve
3	meghívja az email küldő funkciót	sendPasswordResetEmail meghívásának ellenőrzése	—	Email elküldve a helyes címre
4	ugyanazt a választ adja nem létező email esetén	Nem létező email – biztonsági okokból azonos válasz	200	Ugyanaz az üzenet
5	nem küld emailt nem létező felhasználónak	Email küldés nem történik meg	—	sendEmail nem hívódik meg
6	hibát ad vissza hiányzó email esetén	Email mező hiányzik a kérésből	400	"Email is required"
7	nem száll el, ha az email küldés meghiúsul	Email szolgáltatás hiba – graceful handling	200	A kérés nem hiúsul meg
8	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500	"Something went wrong"

4.3 Jelszó visszaállítás – POST /api/auth/reset-password (resetPassword.test.ts)

Fájl: `src/__tests__/api/resetPassword.test.ts`

Cél: Jelszó tényleges visszaállításának tesztelése érvényes tokennel.

#	Teszteset neve	Leírás	Elvárt HTTP státusz	Elvárt válasz

1	sikeresen visszaállítja a jelszót érvényes tokennel	Érvényes, nem lejárt token + új jelszó	200	"Jelszó sikeresen visszaállítva"
2	törli a reset tokent a sikeres visszaállítás után	resetToken és resetTokenExpiry null-ra állítása	—	Token törölve az adatbázisból
3	hibát ad vissza érvénytelen tokennel	Nem létező token megadása	400	"Érvénytelen vagy lejárt visszaállító token"
4	hibát ad vissza hiányzó token esetén	Token mező hiányzik	400	"Token és jelszó megadása kötelező"
5	hibát ad vissza hiányzó jelszó esetén	Jelszó mező hiányzik	400	"Token és jelszó megadása kötelező"
6	hibát ad vissza üres body esetén	Teljesen üres kérés body	400	"Token és jelszó megadása kötelező"
7	a válasz tartalmazza a no-cache fejlécet	Cache-Control: no-store fejléc ellenőrzése	—	no-store fejléc jelen van
8	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500	"Valami hiba történt"

4.4 Profil kezelés – GET & POST /api/profile (profile.test.ts)

Fájl: `src/__tests__/api/profile.test.ts`

Cél: Felhasználói profil lekérdezés és létrehozás/módosítás végpontok tesztelése – autentikáció, jogosultság, validáció, napi célok számítása.

GET /api/profile

#	Teszteset neve	Leírás	Elvárt HTTP státusz
1	visszaadja a profilt bejelentkezett felhasználónak	Bejelentkezett user lekérdezése	200
2	401-es hibát ad vissza, ha nincs bejelentkezve	Munkamenet nélküli kérés	401
3	404-es hibát ad vissza, ha nincs profil	Bejelentkezve, de profil nem létezik	404
4	lekérdezi a profilt az aktív szinttel és céllal együtt	include: activityLevel, goal ellenőrzése	—
5	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500

POST /api/profile

#	Teszteset neve	Leírás	Elvárt HTTP státusz
1	létrehoz egy új profilt	Nem létező profil esetén create hívás	200
2	frissíti a meglévő profilt	Létező profil esetén update hívás	200
3	401-es hibát ad vissza, ha nincs bejelentkezve	Munkamenet nélküli kérés	401
4	403-as hibát ad vissza, ha a userId nem egyezik	Más user profiljának módosítási kísérlete	403

5	400-as hibát ad vissza érvénytelen nemmel	Érvénytelen nem (invalid_gender)	400
6	400-as hibát ad vissza érvénytelen magassággal	Túl alacsony magasság (10 cm)	400
7	400-as hibát ad vissza érvénytelen súllyal	Túl magas súly (600 kg)	400
8	napi célokat számol teljes profil esetén	Teljes profil megadásakor kalória/makró célok számítása	200 + dailyGoals
9	upsert-tel frissíti a napi célokat	dailyGoal.upsert meghívásának ellenőrzése	—

4.5 Aktivitási szintek és célok (activityLevelsAndGoals.test.ts)

Fájl: `src/__tests__/api/activityLevelsAndGoals.test.ts`

Cél: Aktivitási szintek és fitness célok lekérdezése, alapértelmezett adatok létrehozása.

GET /api/activity-levels

#	Teszteset neve	Leírás	Elvárt eredmény
1	visszaadja az aktivitási szinteket	4 aktivitási szint lekérdezése	200, 4 elem
2	id alapján rendezve adja vissza az adatokat	orderBy: { id: 'asc' } ellenőrzése	Rendezett lista
3	Cache-Control fejlécet állít be	s-maxage=3600 fejléc	Cache fejléc jelen van
4	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500

POST /api/activity-levels

#	Teszteset neve	Leírás	Elvárt eredmény
1	létrehozza az alapértelmezett aktivitási szinteket	Ha nem léteznek, létrehozza az alapértelmezetteket	200, létrehozva
2	nem hozza létre újra a már létező szinteket	Ha már léteznek, nem hoz létre duplikátot	200, create nem hívódik

GET /api/goals

#	Teszteset neve	Leírás	Elvárt eredmény
1	visszaadja a célokat	3 cél lekérdezése (Tömegnövelés, Súlytartás, Fogyás)	200, 3 elem
2	id alapján rendezve adja vissza	orderBy: { id: 'asc' } ellenőrzése	Rendezett lista
3	Cache-Control fejlécet állít be	s-maxage=3600 fejléc	Cache fejléc jelen van
4	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500

4.6 Fiók deaktiválás – POST /api/account/deactivate (accountDeactivate.test.ts)

Fájl: `src/__tests__/api/accountDeactivate.test.ts`

Cél: Felhasználói fiók deaktiválási (soft delete) végpont tesztelése.

#	Teszteset neve	Leírás	Elvárt HTTP státusz	Elvárt válasz

1	sikeresen deaktiválja a felhasználói fiókot	Bejelentkezett user fiókjának deaktiválása	200	success: true
2	az isActive mezőt false-ra állítja	Prisma update hívás ellenőrzése	—	isActive: false
3	401-es hibát ad vissza, ha nincs bejelentkezve	Munkamenet nélküli kérés	401	"Nincs bejelentkezve"
4	401-es hibát ad vissza, ha a session nem tartalmaz user id-t	Session user id nélkül	401	"Nincs bejelentkezve"
5	500-as hibát ad vissza adatbázis hiba esetén	DB hiba szimuláció	500	"Nem sikerült deaktiválni a fiókot"

5. tRPC eljárás tesztek

5.1 user.profile eljárás (userProfile.test.ts)

Fájl: `src/__tests__/trpc/userProfile.test.ts`

Cél: A felhasználói profil tRPC eljárás tesztelése – profil létezés, teljességi ellenőrzés, relációk betöltése, szélsőséges esetek.

Profil létezés

#	Teszteset neve	Leírás	Elvárt eredmény
1	should return exists: false when profile does not exist	Nem létező profil lekérdezése	exists: false, isComplete: false, profile: null
2	should return exists: true when profile exists	Létező profil lekérdezése	exists: true, profile nem null

Profil teljesség

#	Teszteset neve	Leírás	Elvárt eredmény
1	should return isComplete: true when all required fields are filled	Minden mező kitöltve	isComplete: true
2	should return isComplete: false when age is missing	age = null	isComplete: false
3	should return isComplete: false when gender is missing	gender = null	isComplete: false
4	should return isComplete: false when heightCm is missing	heightCm = null	isComplete: false
5	should return isComplete: false when weightKg is missing	weightKg = null	isComplete: false
6	should return isComplete: false when activityLevelId is missing	activityLevelId = null	isComplete: false
7	should return isComplete: false when goalId is missing	goalId = null	isComplete: false
8	should return isComplete: false when multiple fields are missing	Minden mező null	isComplete: false

Relációk

#	Teszteset neve	Leírás	Elvárt eredmény
1	should include activityLevel relation when present	activityLevel reláció betöltése	activityLevel objektum jelen van

2	should include goal relation when present	goal reláció betöltése	goal objektum jelen van
3	should include both relations	Mindkét reláció betöltése teljes profilnál	Mindkét objektum jelen van + isComplete: true

Szélsőséges esetek

#	Teszteset neve	Leírás	Elvárt eredmény
1	should handle zero age correctly	age = 0 (falsy érték)	isComplete: false
2	should handle valid minimum age (13)	age = 13	isComplete: true
3	should handle valid maximum age (120)	age = 120	isComplete: true
4	should handle different gender values	male, female, other iteráció	Mind isComplete: true
5	should use correct userId from session	Különböző session userId használata	Helyes userId a lekérdezésben

Adatbázis lekérdezés struktúra

#	Teszteset neve	Leírás	Elvárt eredmény
1	should call findUnique with correct where clause	where: { userId } ellenőrzés	Helyes where feltétel
2	should include activityLevel and goal relations	include paraméter ellenőrzés	Mindkét include: true

3	should only call database once per request	findUnique hívásszám ellenőrzés	Pontosan 1 hívás

6. Segédfüggvény tesztek

6.1 calculateAge – Életkor számítás (calculateHelpers.test.ts)

Fájl: `src/__tests__/helpers/calculateHelpers.test.ts`

Cél: Az életkor kiszámítása születési dátumból. Fix dátumra (2026. február 27.) állított rendszeridővel tesztelve.

#	Teszteset neve	Leírás	Bemeneti dátum	Elvárt életkor
1	helyesen számolja ki az életkort	2000. jan. 1. → 2026. feb. 27.	2000-01-01	26
2	figyelembe veszi, ha még nem volt születésnap	Március 15-i születésnap, most feb. 27.	2000-03-15	25
3	figyelembe veszi, ha már volt születésnap	Január 1-i születésnap, ami már elmúlt	2000-01-01	26
4	helyesen számolja ki a születésnap napján	Pontosan a születésnapon (feb. 27.)	2000-02-27	26
5	0-t ad vissza, ha a mai napon született	Ma született (2026. feb. 27.)	2026-02-27	0
6	72-t ad egy idős személy születési dátumára	1953. június 15.	1953-06-15	72
7	helyesen kezeli a szökőévi születésnapot	Feb. 29-i születésnap (feb. 27 < feb. 29)	2000-02-29	25

6.2 calculateDailyGoals – Napi célok számítás

Cél: Napi kalória- és makrotápanyag-célok kiszámítása a Mifflin-St Jeor egyenlet alapján, figyelembe véve az aktivitási szintet és a fitness célt.

Alkalmazott képlet (Mifflin-St Jeor):
- Férfi: BMR = 10 × súly(kg) + 6.25 × magasság(cm) - 5 × kor + 5
- Nő: BMR = 10 × súly(kg) + 6.25 × magasság(cm) - 5 × kor - 161
- Egyéb: BMR = 10 × súly(kg) + 6.25 × magasság(cm) - 5 × kor - 78

Aktivitási szorzók: Ülő: 1.2 | Enyhén aktív: 1.375 | Mérsékelten aktív: 1.55 | Nagyon aktív: 1.725
Cél módosítók: Fogyás: -500 kcal | Hízás: +500 kcal | Súlytartás: 0

BMR számítás

#	Teszteset neve	Paraméterek	Elvárt eredmény
1	helyes BMR-t számol férfi felhasználónak	25 éves, 180cm, 80kg, ülő, súlytartás	≈ 2166 kcal (BMR=1805, ×1.2)
2	helyes BMR-t számol női felhasználónak	30 éves, 165cm, 60kg, ülő, súlytartás	≈ 1584 kcal (BMR=1320.25, ×1.2)
3	helyes BMR-t számol egyéb nemű felhasználónak	25 éves, 170cm, 70kg, ülő, súlytartás	≈ 1871 kcal (BMR=1559.5, ×1.2)

Aktivitási szint szorzók

#	Teszteset neve	Szorzó	Elvárt kalória
1	ülő életmódnál 1.2-es szorzót alkalmaz	1.2	round(1805 × 1.2) = 2166
2	enyhén aktívnál 1.375-ös szorzót alkalmaz	1.375	round(1805 × 1.375) = 2482
3	mérsékelten aktívnál 1.55-ös szorzót alkalmaz	1.55	round(1805 × 1.55) = 2798

4	nagyon aktívnál 1.725-ös szorzót alkalmaz	1.725	round(1805 × 1.725) = 3114
5	ismeretlen szintnél alapértelmezetten 1.2-t alkalmaz	1.2 (alapértelmezett)	round(1805 × 1.2) = 2166

Cél alapján kalória módosítás

#	Teszteset neve	Módosítás	Elvárt kalória
1	fogyásnál 500 kalóriát levon	TDEE - 500	round(2166 - 500) = 1666
2	hízásnál 500 kalóriát hozzáad	TDEE + 500	round(2166 + 500) = 2666
3	súlytartásnál nem módosítja	TDEE + 0	2166

Minimális kalória

#	Teszteset neve	Minimum érték
1	férfiaknál nem csökken 1500 alá	≥ 1500 kcal
2	nőknél nem csökken 1200 alá	≥ 1200 kcal
3	egyéb nemnél nem csökken 1350 alá	≥ 1350 kcal

Makró számítások és visszatérési érték

#	Teszteset neve	Leírás
1	a zsír a kalóriák 30%-a kell legyen	fatGoal = round(calories × 0.30 / 9)
2	makrók összege ≈ teljes kalória	protein×4 + fat×9 + carbs×4 ≈ calories (± 10 kcal)

3	aktívabb életmódnál magasabb fehérje	Nagyon aktív > ülő fehérje cél
4	fogyásnál magasabb fehérje szorzó	Fogyás > súlytartás fehérje
5	hízásnál magasabb fehérje szorzó	Hízás > súlytartás fehérje
6	kerekített számokat ad vissza	Minden érték egész szám
7	tartalmazza az összes szükséges mezőt	caloriesGoal, proteinGoal, fatGoal, carbsGoal
8	pozitív értékeket ad vissza	Minden érték > 0

Realisztikus esetek

#	Teszteset neve	Profil	Elvárt tartomány
1	20 éves aktív férfi – izomépítés	185cm, 85kg, nagyon aktív, hízás	> 3000 kcal, > 150g fehérje
2	50 éves ülő életmódú nő – fogyás	160cm, 75kg, ülő, fogyás	1200–2000 kcal
3	35 éves mérsékelten aktív férfi – súlytartás	175cm, 75kg, mérsékelt, tartás	2200–3200 kcal

7. Komponens tesztek (UI)

A komponens tesztek a **React Testing Library**-t használják JSDOM környezetben. Mock-olt next-auth, next/navigation, next/image és tRPC modulokkal dolgoznak.

7.1 Header komponens (Header.test.tsx)

Fájl: `src/__tests__/components/Header.test.tsx`

Cél: A fejléc komponens tesztelése – logó, navigáció, bejelentkezési állapot, jogosultságkezelés, mobilmenü, sticky pozícionálás.

#	Teszteset neve	Leírás	Elvárt eredmény
1	megjeleníti a header komponenst	Header elem renderelése	**<header> jelen van**
2	hibamentesen renderelődik bejelentkezett felhasználóval	Authenticated session-nel	**Nincs hiba**
3	megjeleníti a Corelytics logót	Logó kép megjelenítése	**Logó alt szöveg megtalálható**
4	megjeleníti a navigációs linkeket	Ételek, Edzések, Konklúzió linkek	**Mindhárom link jelen van**
5	megjeleníti a bejelentkezés linket, ha nincs bejelentkezve	Kijelentkezett állapot	**"Bejelentkezés" link megjelenik**
6	megjeleníti a felhasználó nevét bejelentkezés után	Bejelentkezett állapot	**Felhasználónév megjelenik**
7	megjeleníti a felhasználói menü gombot	Bejelentkezett felhasználó menügombja	**Menügomb a névvel**
8	nem jeleníti meg az Admin Panel linket normál felhasználónak	permissionLevel: 0	**Nincs admin link**
9	hibamentesen renderelődik moderátor jogosultsággal	permissionLevel: 1	**Nincs hiba**
10	mobilmenüben megjelenik az Admin Panel link moderátornak	permissionLevel: 2, mobilmenü megnyitás	**"Admin Panel" megjelenik**
11	a navigációs linkek helyes href-ekkel rendelkeznek	/#meals, /#workouts, /#conclusion	**Helyes href-ek**

12	sticky pozícionálással rendelkezik	CSS osztályok ellenőrzése	**sticky + top-0 osztályok**

7.2 Footer komponens (Footer.test.tsx)

Fájl: `src/__tests__/components/Footer.test.tsx`

Cél: A lábléc komponens tesztelése – céginfo, gyors linkek, kapcsolat, copyright, bejelentkezési állapot kezelés.

#	Teszteset neve	Leírás	Elvárt eredmény
1	renders footer component	Footer elem renderelése	**<footer> elem**
2	renders without crashing	Hibamentes renderelés	**Nincs hiba**
3	renders company info section	"Corelytics" név és leírás	**Céginfo szekció**
4	renders quick links section	Ételek, Edzések, Konklúzió linkek	**Gyors linkek szekció**
5	renders contact info section	Email és közösségi média szöveg	**Kapcsolat szekció**
6	renders copyright with current year	© {aktuális év} Corelytics	**Dinamikus évszám**
7	renders login link when not authenticated	Kijelentkezett állapot	**"Bejelentkezés" link**
8	renders logout button when authenticated	Bejelentkezett állapot	**"Kijelentkezés" gomb**

9	calls signOut when logout button is clicked	Kijelentkezés gomb kattintás	**signOut({ callbackUrl: '/' })**
10	has correct link hrefs	/#meals, /#workouts, /#conclusion	**Helyes href-ek**
11	login link has correct href	/auth/signin	**Helyes href**

7.3 Főoldal – Home (Home.test.tsx)

Fájl: `src/__tests__/components/Home.test.tsx`

Cél: A főoldal tesztelése – szekciók megjelenítése, sorrend, header/footer jelenlét.

#	Teszteset neve	Leírás	Elvárt eredmény
1	hibamentesen renderelődik	Alapvető renderelés	**Nincs hiba**
2	megjeleníti a főcímet	"Gondold újra az edzésterved"	**Főcím szöveg**
3	megjeleníti a Corelytics logót	Logó kép a főoldalon	**≥ 1 logó**
4	megjeleníti az ételek szekciót	#meals szekció + "Étrend- és edzéstervező alkalmazás"	**Szekció + szöveg**
5	megjeleníti az edzések szekciót	#workouts szekció + "Edzésterv nyilvántartása"	**Szekció + szöveg**
6	megjeleníti a konklúzió szekciót	#conclusion szekció + "Súly- és testösszetétel nyomon követés"	**Szekció + szöveg**
7	megjeleníti a Header komponenst	<header> elem jelenlét	**Header megjelenik**

8	megjeleníti a Footer komponenst	<footer> elem jelenlét	**Footer megjelenik**
9	tartalmazza a három fő szekciót a helyes sorrendben	meals → workouts → conclusion sorrend	**Helyes sorrend**
10	bejelentkezett felhasználóval is hibamentesen renderelődik	Authenticated session-nel	**Nincs hiba**

7.4 Bejelentkezés oldal – SignIn (SignIn.test.tsx)

Fájl: `src/__tests__/components/SignIn.test.tsx`

Cél: A bejelentkezési oldal tesztelése – űrlap megjelenítés, interakció, hitelesítés, hibaüzenetek.

#	Teszteset neve	Leírás	Elvárt eredmény
1	hibamentesen renderelődik	Alapvető renderelés	**Nincs hiba**
2	megjeleníti a bejelentkezési címet	"Jelentkezz be a fiókodba"	**Cím megjelenik**
3	megjeleníti az email és jelszó mezőket	Email és jelszó beviteli mezők	**Mindkét mező jelen van**
4	megjeleníti a bejelentkezés gombot	"Bejelentkezés" gomb	**Gomb megjelenik**
5	megjeleníti az elfelejtett jelszó linket	/auth/forgot-password link	**Link helyes href-fel**
6	megjeleníti a regisztrációs linket	/auth/signup link	**Link helyes href-fel**

7	megjeleníti a "Még nincs fiókod?" szöveget	Regisztrációra ösztönző szöveg	**Szöveg megjelenik**
8	az email mező type attribútuma email	type="email"	**Helyes type**
9	a jelszó mező type attribútuma password	type="password"	**Helyes type**
10	megjeleníti a Header és Footer komponenseket	header + footer elem jelenlét	**Mindkettő megjelenik**
11	mutat hibaüzenetet érvénytelen bejelentkezésnél	signIn → error: 'CredentialsSignin'	**"Érvénytelen bejelentkezési adatok"**
12	meghívja a signIn funkciót az űrlap beküldésekor	Email + jelszó kitöltés, gomb kattintás	**signIn('credentials', { ... })**

7.5 Regisztráció oldal – SignUp (SignUp.test.tsx)

Fájl: `src/__tests__/components/SignUp.test.tsx`

Cél: A regisztrációs oldal tesztelése – űrlap megjelenítés, jelszó-egyeztetés, API hívás, átirányítás, hibaüzenetek.

#	Teszteset neve	Leírás	Elvárt eredmény
1	hibamentesen renderelődik	Alapvető renderelés	**Nincs hiba**
2	megjeleníti a regisztrációs címet	"Hozd létre a fiókodat"	**Cím megjelenik**
3	megjeleníti az összes beviteli mezőt	Név, email, jelszó, jelszó megerősítés	**Mind a 4 mező**

4	megjeleníti a regisztráció gombot	"Regisztráció" gomb	**Gomb megjelenik**
5	megjeleníti a bejelentkezés linket	/auth/signin link	**Link helyes href-fel**
6	a jelszó mezők type attribútuma password	Mindkét jelszó mező type="password"	**Helyes type-ok**
7	az email mező type attribútuma email	type="email"	**Helyes type**
8	megjeleníti a Header és Footer komponenseket	header + footer elem jelenlét	**Mindkettő megjelenik**
9	hibaüzenetet mutat, ha a jelszavak nem egyeznek	Különböző jelszó a két mezőben	**"A jelszavak nem egyeznek"**
10	nem hívja meg az API-t, ha a jelszavak nem egyeznek	Kliensoldali validáció megakadályozza	**axios.post nem hívódik**
11	sikeres regisztráció után átirányít a bejelentkezés oldalra	API 200 → router.push('/auth/signin')	**Átirányítás megtörténik**
12	hibaüzenetet mutat API hiba esetén	API 400 → hibaüzenet megjelenítése	**"Ez az email cím már használatban van"**
13	a gomb kikapcsolt állapotba kerül betöltés közben	API hívás közben disabled állapot	**Gomb disabled**

8. Összesített statisztika

Teszt fájl	Kategória	Tesztek száma	Fájl elérési út

envSchema.test.ts	Validáció	21	src/__tests__/validation/
profileSchema.test.ts	Validáció	20	src/__tests__/validation/
foodLog.test.ts	Validáció	26	src/__tests__/validation/
foodCreate.test.ts	Validáció	30	src/__tests__/validation/
register.test.ts	API	7	src/__tests__/api/
forgotPassword.test.ts	API	8	src/__tests__/api/
resetPassword.test.ts	API	8	src/__tests__/api/
profile.test.ts	API	14	src/__tests__/api/
activityLevelsAndGoals.test.ts	API	10	src/__tests__/api/
accountDeactivate.test.ts	API	5	src/__tests__/api/
userProfile.test.ts	tRPC	21	src/__tests__/trpc/
calculateHelpers.test.ts	Segédfüggvény	24	src/__tests__/helpers/
Header.test.tsx	Komponens	12	src/__tests__/components/
Footer.test.tsx	Komponens	11	src/__tests__/components/
Home.test.tsx	Komponens	10	src/__tests__/components/

SignIn.test.tsx	Komponens	12	src/__tests__/components/
SignUp.test.tsx	Komponens	13	src/__tests__/components/
ÖSSZESEN		**252**	

Tesztlefedettség kategóriánként

Kategória	Tesztek száma	Arány	Lefedett terület
Validáció (Zod sémák)	97	38.5%	Környezeti változók, profil, étkezés napló, étel létrehozás
API végpontok	52	20.6%	Regisztráció, jelszókezelés, profil, aktivitás, deaktiválás
tRPC eljárások	21	8.3%	Felhasználói profil lekérdezés és teljességi ellenőrzés
Segédfüggvények	24	9.5%	Életkor számítás, napi kalória/makró célok
UI komponensek	58	23.0%	Header, Footer, Főoldal, Bejelentkezés, Regisztráció

Megjegyzés: Ez a dokumentum a `src/__tests__/` mappában található összes tesztfájl alapján készült. A tesztek a Jest teszt keretrendszert és a React Testing Library-t használják. Az API és tRPC tesztek mock-olt Prisma adatbázissal és NextAuth munkamenettel dolgoznak, nem igényelnek valós adatbázis kapcsolatot.